



ESOF Metro Campus

School of Computing

Bachelor of Information Technology Honours

Batch 002 - Level 04 – Semester 02

ECU4315– Data Science

Assignment 01

Reg No: E123462

Name: W.A.P.D. Hansika

September 2024

Table of Contents

Linear Regression	1
Task 1 - Edit and Clean the Dataset	1
Task 2 - Visualize the Dataset	7
Task 3 - Create and Train a Linear Regression Model	11
Task 4 - Make Predictions	15
Full Code	17

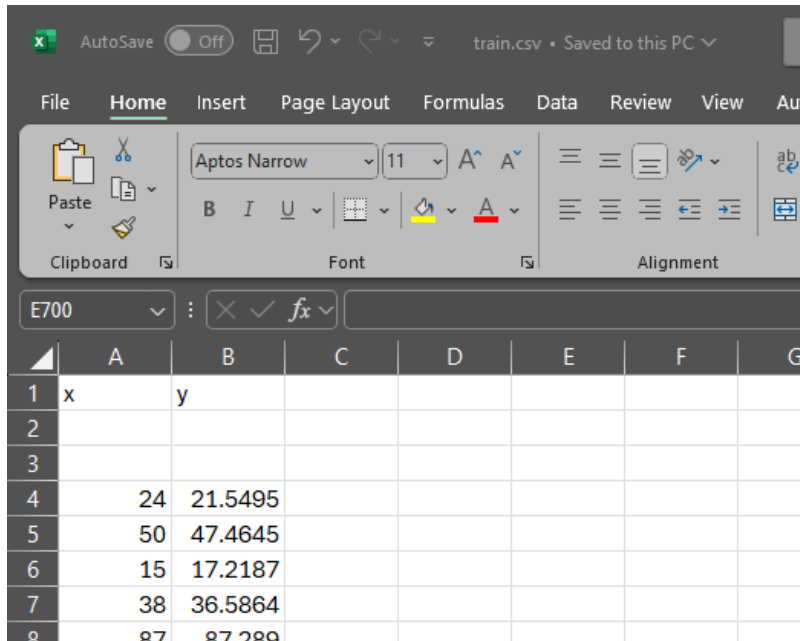
Linear Regression

Expected to create a linear regression model using python for the given data set and train the model using the train.csv dataset and test the model using test.csv dataset.

Task 1 - Edit and Clean the Dataset

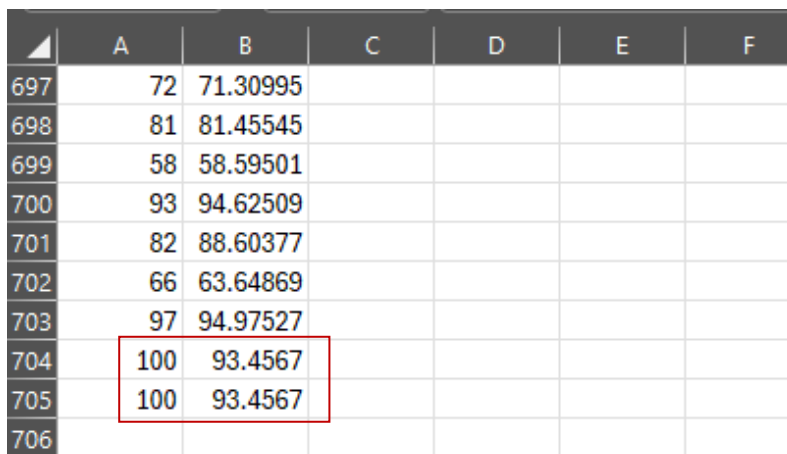
First, manually edit the train.csv file to add two empty spaces and two duplicate records, and then clean up the train.csv dataset. Sample values can be given when filling in the blanks.

Step 01 - Add two empty spaces.



	A	B	C	D	E	F	G
1	x	y					
2							
3							
4	24	21.5495					
5	50	47.4645					
6	15	17.2187					
7	38	36.5864					
8	37	37.789					

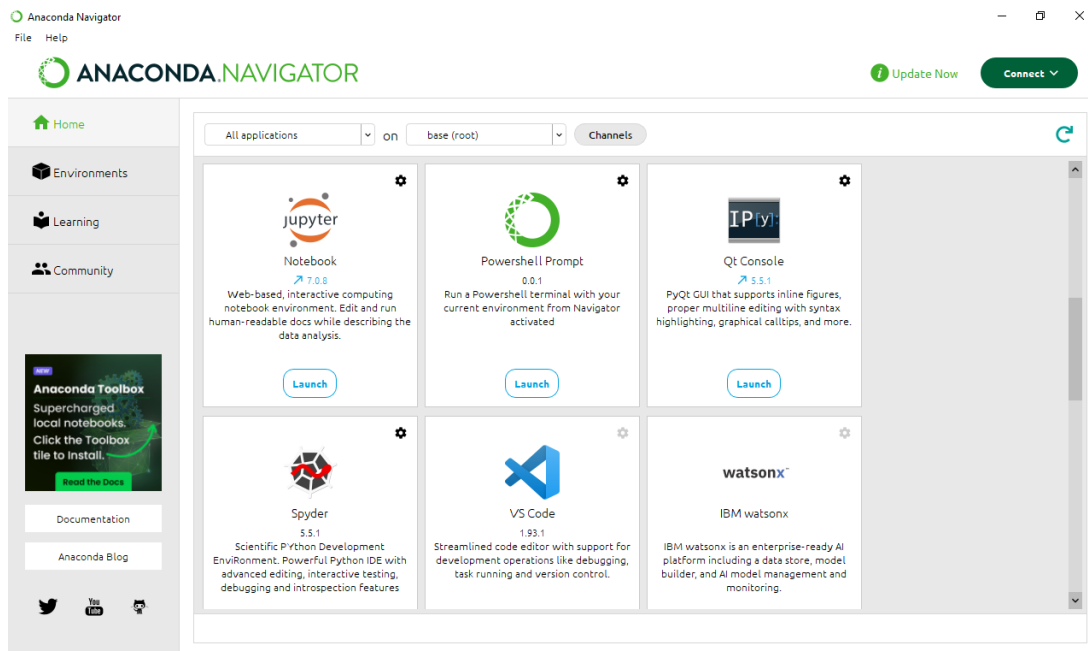
Step 02 - Add two duplicate records.



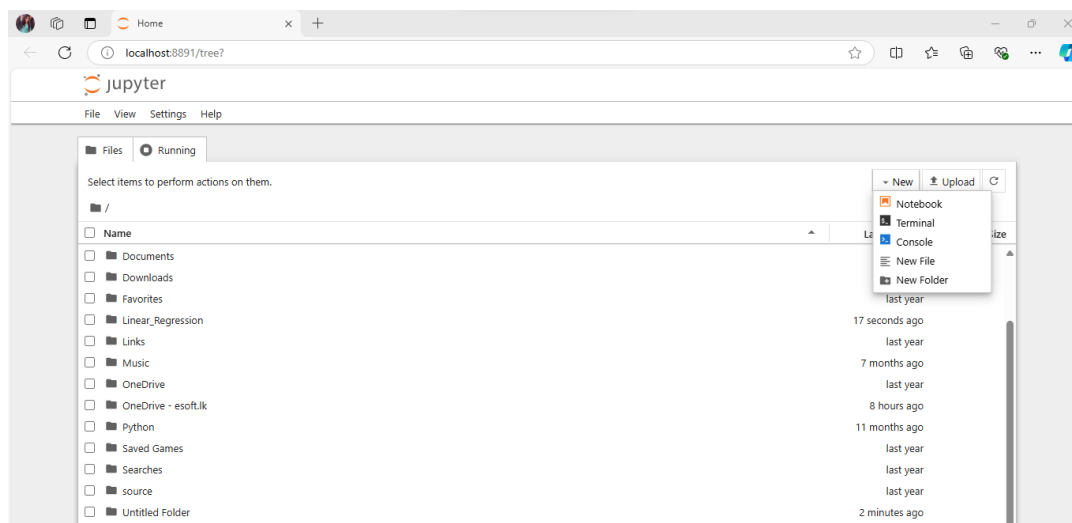
	A	B	C	D	E	F
697	72	71.30995				
698	81	81.45545				
699	58	58.59501				
700	93	94.62509				
701	82	88.60377				
702	66	63.64869				
703	97	94.97527				
704	100	93.4567				
705	100	93.4567				
706						

Now, Clean the train.csv dataset.

Step 03 - Open The Anaconda Software and Go top Jupyter Notebook.



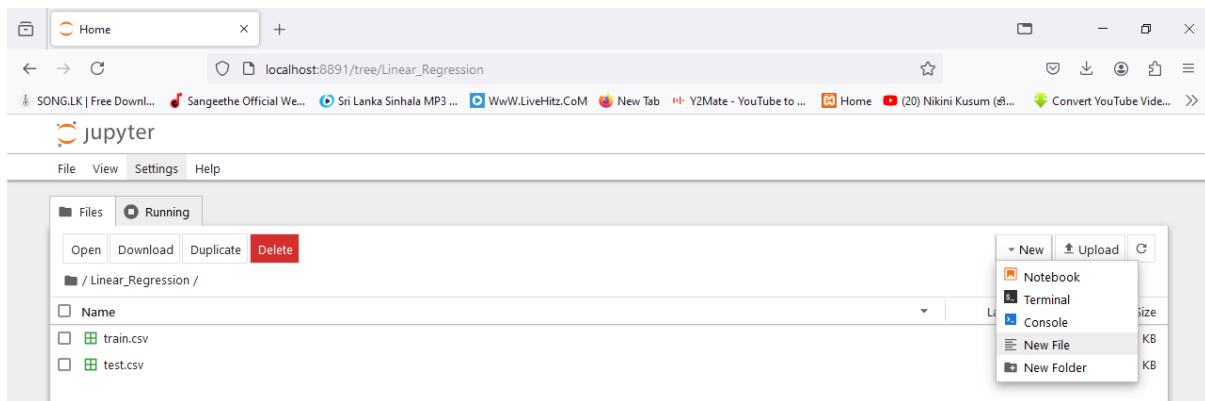
Step 04 - Create New Folder and Rename “Linear_Regression”.



Step 05 – Open Folder Upload the csv Files.



Step 06 – Go to Notebook and Create Python File. After then open Python File.



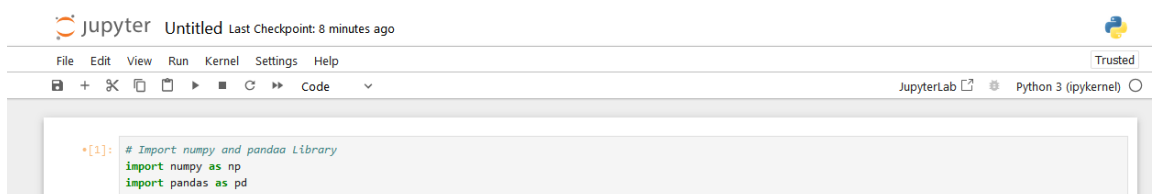
Step 07 – Using the panda's library to read an csv file called "train.csv" into a DataFrame and then printing that DataFrame. This imports the NumPy and Pandas libraries, allowing to use their functions.

```
import numpy as np
```

NumPy (Numerical Python) is a library for numerical computing in Python. It provides support for arrays, matrices, and a wide range of mathematical functions to operate on these data structures.

```
import pandas as pd
```

Pandas is a powerful library for data manipulation and analysis. It provides data structures like Data Frames and Series, which are built on top of NumPy.



Step 08 – Read the csv files.

```
[1]: # Import numpy and pandas Library
import numpy as np
import pandas as pd
```

```
[3]: #Load the train dataset
df1=pd.read_csv("train.csv")
print(df1)
```

	x	y
0	NaN	NaN
1	NaN	NaN
2	24.0	21.549452
3	50.0	47.464463
4	15.0	17.218656
...	...	...
699	82.0	88.603770
700	66.0	63.648685
701	97.0	94.975266
702	100.0	93.456700
703	100.0	93.456700

[704 rows x 2 columns]

```
[5]: #Load the test dataset
df2=pd.read_csv("test.csv")
print(df2)
```

```

      x      y
0    77  79.775152
1    21  23.177279
2    22  25.609262
3    20  17.857388
4    36  41.849864
..    ..    ...
295  71  68.545888
296  46  47.334876
297  55  54.090637
298  62  63.297171
299  47  52.459467
```

```
[300 rows x 2 columns]
```

`pd.read_csv()` is a function that reads an csv file and loads it into a Pandas DataFrame. The `df1` and `df2` variables will now contain all the data from the csv file, structured in a way that's easy to manipulate and analyze.

This prints the contents of the DataFrame to the console, so you can see the data you loaded from the csv file.

Step 09 - Breakdown of the Code

Check for Missing Values. This shows the number of missing values in each column of the original DataFrame `df1`.

```
[4]: #Check train.csv Missing Values
df1.isnull().sum()
```

```
[4]: x      2
     y      3
     dtype: int64
```

Step 10 – Filling the missing values

```
dtype: int64
```

```
[100]: df1.loc[0,'x'] = 25 # Replace 'x' with the actual column name for x
df1.loc[0,'y'] = 23.456785 # Replace 'y' with the actual column name for y
df1.loc[1,'x'] = 56 # Assuming this is the second missing value for x
df1.loc[1,'y'] = 45.345665 # Assuming this is the second missing value for y

# Alternatively, you can use this if you know the exact index positions of missing values
# df1.loc[[index_1, index_2], ['x', 'y']] = [[24, 23.456785], [56, 45.345665]]

# Save the updated DataFrame (optional)
df1.to_csv('train_filled.csv', index=False)

# Display the updated DataFrame
print(df1)
```

```

      x      y
0    25.0  23.456785
1    56.0  45.345665
2    24.0  21.549452
3    50.0  47.464463
4    15.0  17.218656
..    ...    ...
699  82.0  88.603770
700  66.0  63.648685
701  97.0  94.975266
702 100.0  93.456700
703 100.0  93.456700
```

```
[704 rows x 2 columns]
```

This code is used to fill missing values in specific cells of a DataFrame ('df1') by directly assigning values. The 'loc[]' function allows accessing specific rows and columns in the DataFrame.

In the first line, 'df1.loc[0, 'x'] = 25', it accesses the first row (index '0') in the column 'x' and assigns it the value '25'. The next line, 'df1.loc[0, 'y'] = 23.456785', targets the same row but in the column 'y', assigning it the value '23.456785'. Similarly, the code updates the second row (index '1') by assigning '56' to column 'x' and '45.345665' to column 'y'. These assignments directly modify the DataFrame in place, filling in the missing values.

The alternative code provided as a comment shows how to assign values to multiple rows at once, using a list of index positions and a list of value pairs for the columns 'x' and 'y'. The last part of the code saves the updated DataFrame to a CSV file named 'train_filled.csv', and then prints the updated DataFrame to the console.

```
[103]: #Check test.csv Missing Values
      df2.isnull().sum()

[103]: x    0
      y    0
      dtype: int64
```

Step 11 - Drop Missing Values

```
[101]: # Drop train.csv Missing Value
      df3=df1.dropna()

[102]: df3.isnull().sum()

[102]: x    0
      y    0
      dtype: int64
```

The code drops any rows with missing values from the DataFrame 'df1' using 'dropna()' and stores the cleaned data in 'df3'. It then checks if there are any remaining missing values in 'df3' by counting them with 'isnull().sum()'. If all missing values were successfully removed, the count for each column should be zero.

Step 12 - Check for Duplicates

```
--
[104]: #Check for Duplicate values
      df1.duplicated().sum()

[104]: 1
```

The code 'df1.duplicated().sum()' is used to check for duplicate rows in the DataFrame 'df1'. It identifies any rows that are identical to previous ones and returns the total count of these duplicates. If the output is 0, it means there are no duplicate rows in the dataset.

The code first removes duplicate rows from the DataFrame `df1` using `drop\_duplicates()`, resulting in a new DataFrame `df4` that contains only unique rows. After the duplicates are removed, the `df4.duplicated().sum()` function is used to verify whether any duplicates remain in `df4`. Ideally, this should return zero, indicating that all duplicate entries have been successfully eliminated from the dataset. This process helps ensure the data is clean and free from redundancy.

```
[104]: #Check for Duplicate values  
df1.duplicated().sum()
```

```
[104]: 1
```

```
[105]: # Drop Duplicate values  
df4=df1.drop_duplicates()
```

```
[106]: df4.duplicated().sum()
```

```
[106]: 0
```

```
[107]: #Check for Duplicate values  
df2.duplicated().sum()
```

```
[107]: 0
```

The code first removes duplicate rows from the DataFrame `df1` using `drop\_duplicates()`, resulting in a new DataFrame `df4` that contains only unique rows. After the duplicates are removed, the `df4.duplicated().sum()` function is used to verify whether any duplicates remain in `df4`. Ideally, this should return zero, indicating that all duplicate entries have been successfully eliminated from the dataset. This process helps ensure the data is clean and free from redundancy.

Task 2 - Visualize the Dataset

Step 13 – Import Matplotlib

```
[108]: # Import Matplotlib
import matplotlib.pyplot as plt
```

The line `import matplotlib.pyplot as plt` imports the Matplotlib library's `pyplot` module, which is commonly used for creating static, interactive, and animated visualizations in Python. By convention, it is imported as `plt` to simplify the syntax when calling its functions. This module provides various functions for plotting graphs, such as line plots, bar charts, histograms, and scatter plots, allowing for effective data visualization. Once imported, you can use functions like `plt.plot()`, `plt.show()`, and others to create and customize your plots easily.

Step 14 - Extracting Values from DataFrames for Model Training

```
[140]: #train x
x_train=df1["x"].values

print(x_train)
```

```
[2.50000000e+01 5.60000000e+01 2.40000000e+01 5.00000000e+01
 1.50000000e+01 3.80000000e+01 8.70000000e+01 3.60000000e+01
 1.20000000e+01 8.10000000e+01 8.10000000e+01 5.00000000e+00
 1.60000000e+01 1.60000000e+01 2.40000000e+01 3.90000000e+01
 5.40000000e+01 6.00000000e+01 2.60000000e+01 7.30000000e+01
 2.90000000e+01 3.10000000e+01 6.80000000e+01 8.70000000e+01
 5.80000000e+01 5.40000000e+01 8.40000000e+01 5.80000000e+01
 4.90000000e+01 2.00000000e+01 9.00000000e+01 4.80000000e+01
 4.00000000e+00 4.00000000e+00 4.20000000e+01 0.00000000e+00
 6.00000000e+01 9.30000000e+01 3.90000000e+01 7.00000000e+00
 2.10000000e+01 6.80000000e+01 8.40000000e+01 0.00000000e+00
 5.80000000e+01 1.90000000e+01 3.60000000e+01 1.90000000e+01
 5.90000000e+01 5.10000000e+01 1.90000000e+01 3.30000000e+01
 8.50000000e+01 4.40000000e+01 5.00000000e+00 5.90000000e+01
 1.40000000e+01 9.00000000e+00 9.00000000e+00 6.90000000e+01
 1.00000000e+01 1.70000000e+01 5.80000000e+01 7.40000000e+01
 2.10000000e+01 5.10000000e+01 1.90000000e+01 5.00000000e+01
 2.40000000e+01 0.00000000e+00 1.20000000e+01 1.20000000e+01]
```

```
[141]: #train y
y_train=df1["y"].values

print(y_train)
```

```
[ 23.456785  45.345665  21.54945196  47.46446305  17.21865634
 36.58639803  87.28898389  32.46387493  10.78089683  80.7633986
 24.61215147   6.96331907  11.23757338  13.53290206  24.60323899
 39.40049976  48.43753838  61.69900319  26.92832418  70.4052055
 29.34092408  25.30895192  69.02934339  84.99484703  57.04310305
 50.5921991  83.02772202  57.05752706  47.95883341  24.34226432
 94.68488281  48.03970696   7.08132338  21.99239907  42.33151664
   0.32908944  61.92303698  91.17716423  39.45358014   5.99606961
 22.59015942  61.18044414  85.02778957  -1.28631089  61.94273962
 21.96033347  33.66194193  17.60946242  58.5630564  52.82390762
 22.1363481  35.07467353  86.18822311  42.63227697   4.09817744
 61.2229864  17.70677576  11.85312574  80.23051695  62.64931741
  9.6168598  20.02797699  61.7510743  71.61010303  23.77154623
 51.90142035  22.66073682  50.02897927  26.68794368   0.3769119
   6.806419  77.33986001  28.90260209  66.7346608   0.70751064
 57.07748383  28.41453196  44.46272123   7.459606   2.31670811
   4.92854619  52.50336074  91.19109623   8.48916433   6.96337197
 31.97989959  81.4281205  22.62365422  78.52505087  25.80714057]
```

This code snippet focuses on preparing the training data for a machine learning model by extracting feature and target values from the DataFrame `df1`. It first retrieves the values from the column labeled `x` and assigns them to the variable `x\_train` as a NumPy array, which will be used as input features for the model. Similarly, it extracts values from the column `y` and stores them in `y\_train`, representing the corresponding target variable. Finally, the code prints both arrays to the console, allowing for verification of the data extraction process. This setup is essential for training the model effectively, ensuring the features and targets are appropriately aligned.

Step 15 - - Extracting Values from DataFrames for Model Test

```
[142]: # test x
x_test=df2["x"].values

print(x_test)
```

77	21	22	20	36	15	62	95	20	5	4	19	96	62	36	15	65	14
87	69	89	51	89	27	97	58	79	21	93	27	99	31	33	80	28	47
53	69	28	33	91	71	50	76	4	37	70	68	40	35	94	88	52	31
59	0	39	64	69	57	13	72	76	61	82	18	41	50	55	13	46	13
79	53	15	28	81	69	52	84	68	27	56	48	40	39	82	100	59	43
67	38	63	91	60	14	21	87	73	32	2	82	19	74	42	12	1	90
89	0	41	16	94	97	66	24	17	90	13	0	64	96	98	12	41	47
78	20	89	29	64	75	12	25	28	30	65	59	64	53	71	97	73	9
12	63	99	60	35	2	60	32	94	84	63	22	81	93	33	7	42	46
54	16	49	43	95	66	21	35	80	37	54	56	1	32	58	32	46	72
17	97	93	91	37	4	54	51	27	46	92	73	77	91	61	99	4	72
19	57	78	26	74	90	66	13	40	77	67	75	23	45	59	44	23	55
55	95	12	4	7	100	48	42	96	39	100	87	14	14	37	5	88	91
65	74	56	16	5	28	92	46	54	39	44	31	68	86	90	38	21	95
56	60	65	78	89	6	67	36	16	100	45	73	57	20	76	34	55	72
55	8	56	72	58	6	96	23	58	23	19	25	64	21	59	19	16	42
43	61	92	11	41	1	8	71	46	55	62	47]						

```
[143]: # test y
y_test=df2["y"].values

print(y_test)
```

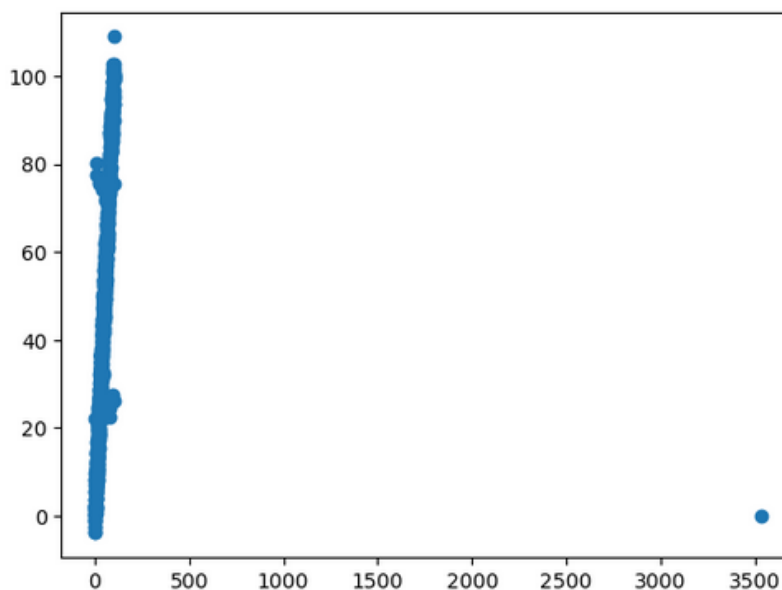
79.77515201	23.17727887	25.60926156	17.85738813	41.84986439
9.80523488	58.87465933	97.61793701	18.39512747	8.74674765
2.81141583	17.09537241	95.14907176	61.38800663	40.24701716
14.82248589	66.95806869	16.63507984	90.65513736	77.22982636
92.11906278	46.91387709	89.82634442	21.71380347	97.41206981
57.01631363	78.31056542	19.1315097	93.03483388	26.59112396
97.55155344	31.43524822	35.12724777	78.61042432	33.07112825
51.69967172	53.62235225	69.46306072	27.42497237	36.34644189
95.06140858	68.16724757	50.96155532	78.04237454	5.60766487
36.11334779	67.2352155	65.01324035	38.14753871	34.31141446
95.28503937	87.84749912	54.08170635	31.93063515	59.61247085
-1.04011421	47.49374765	62.60089773	70.9146434	56.14834113
14.05572877	68.11367147	75.59701346	59.225745	85.45504157
17.76197116	38.68888682	50.96343637	51.83503872	17.0761107
46.56141773	10.34754461	77.91032969	50.17008622	13.25690647
31.32274932	73.9308764	74.45114379	52.01932286	83.68820499
70.3698748	23.44479161	49.83051801	49.88226593	41.04525583
33.37834391	81.29750133	105.5918375	56.82457013	48.67252645
67.02150613	38.43076389	58.61466887	89.12377509	60.9105427
13.83959878	16.89085185	84.06676818	70.34969772	33.38474138
-1.63296825	88.54475895	17.44047622	75.69298554	41.97607107
12.59244741	0.27530726	98.13258005	87.45721555	-2.34473854
39.3294153	16.68715211	96.58888601	97.70342201	67.01715955
25.63476257	13.41310757	95.15647284	9.74416426	-3.46788379

This code snippet prepares the testing data for evaluation after training a machine learning model. It begins by extracting the values from the column labeled 'x' in the DataFrame 'df2' and storing them in the variable 'x_test' as a NumPy array, which serves as the input features for the test set. It then retrieves the values from the column 'y' and assigns them to 'y_test', representing the corresponding target variable for the test data. Finally, the code prints both 'x_test' and 'y_test' arrays to the console for verification. This step is crucial as it ensures that the model will be tested against the correct input and output values, enabling accurate performance evaluation.

Step 16 – This code creates a scatter plot using Matplotlib to visualize the relationship between the training data variables 'x_train' and 'y_train'.

```
[144]: # Scatter Plot
plt.scatter(x_train, y_train)

plt.show()
```



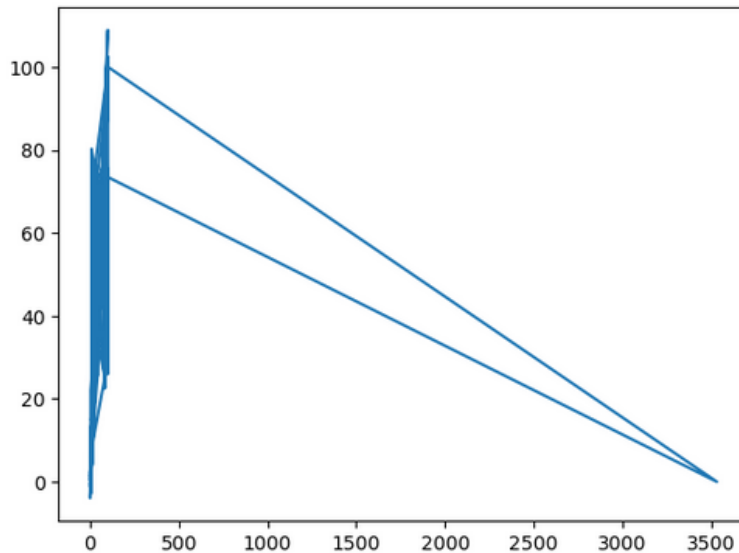
Scatter Plot Creation - The 'plt.scatter(x_train, y_train)' function plots individual data points on a two-dimensional graph, where each point represents a pair of 'x' (independent variable) and 'y' (dependent variable) values from the training dataset. This allows for an easy visual inspection of how the two variables correlate.

Display the Plot - The 'plt.show()' function then displays the scatter plot in a new window. This visual representation helps in understanding the distribution, trends, and potential relationships within the training data, which can be valuable for further analysis and model training decisions.

Step 17 – This code generates a line plot using Matplotlib to visualize the relationship between the training data variables `x\_train` and `y\_train`.

```
[145]: # Line Plot
plt.plot(x_train, y_train)

plt.show()
```



nects the data points of `y\_train` against `x\_train` with straight lines. This type of visualization is particularly useful for observing trends and patterns in the data, making it easier to identify how changes in the independent variable (`x\_train`) affect the dependent variable (`y\_train`).

Display the Plot - The `plt.show()` function is used to display the line plot in a separate window. Unlike a scatter plot, which focuses on individual data points, the line plot emphasizes the overall trend and continuity of the data, providing a clearer picture of how the dependent variable behaves across the range of the independent variable.

Task 3 - Create and Train a Linear Regression Model

Step 18

```
[146]: from sklearn.model_selection import train_test_split
```

The provided code snippet includes essential imports from the Scikit-learn library, which is widely used for machine learning tasks in Python. The first import, `train\_test\_split`, allows for the division of a dataset into training and testing subsets. This is a crucial step in the machine learning workflow, as it enables the model to be trained on one set of data while being evaluated on a separate set. This approach helps ensure that the model generalizes well to unseen data, reducing the risk of overfitting, where the model performs well on training data but poorly on new, unseen data.

```
[147]: from sklearn.linear_model import LinearRegression
```

The second important import is the `LinearRegression` class, which provides the functionality to create a linear regression model. Linear regression is a fundamental statistical method used to establish a relationship between dependent and independent variables. By training this model on the training data, you can predict the values of the dependent variable based on the given independent variables. This technique is particularly effective when the relationship between variables is linear, making it a go-to choice for many regression problems.

```
[148]: from sklearn.metrics import mean_squared_error, r2_score
```

Lastly, the code includes imports for evaluating the model's performance, specifically `mean\_squared\_error` and `r2\_score`. The `mean\_squared\_error` function quantifies the average squared differences between the predicted values and the actual values, providing a measure of the model's accuracy; a lower mean squared error indicates better performance. Meanwhile, `r2\_score` measures how well the model explains the variability of the dependent variable, with values ranging from 0 to 1. A score closer to 1 suggests a strong explanatory power of the model, making these metrics vital for assessing the quality and effectiveness of the regression analysis.

Step 19 - Prepare the data

```
[149]: # Prepare the data
x = df1[['x']]
y = df1['y']
```

The code snippet prepares the data for a linear regression model by isolating the independent and dependent variables. In this case, the variable `x` is defined as a DataFrame containing the column `x` from `df1`, which serves as the feature or independent variable.

By using double square brackets `[['x']]`, the resulting variable `x` remains a DataFrame rather than a Series, which is important for compatibility with Scikit-learn's model fitting methods.

The variable `y`, on the other hand, is defined as a Series containing the values of the dependent variable `y` from the same DataFrame, `df1`. In a linear regression context, `y` represents the target variable that the model aims to predict based on the values in `x`. By structuring the data this way, you set the stage for the subsequent steps in the machine learning process, including model training and evaluation. This data preparation is crucial, as it ensures that the model has the correct inputs to learn from when predicting outcomes.

Step 20 - Split the data into training and testing sets

```
[150]: # Split the data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

This code snippet utilizes the `train\_test\_split` function from Scikit-learn to divide the dataset into training and testing sets. It takes the independent variable `x` and the dependent variable `y` as inputs and specifies a `test\_size` of 0.2, meaning 20% of the data will be allocated for testing while the remaining 80% will be used for training.

The `random\_state` parameter is set to 42 to ensure the results are reproducible, meaning that every time the code is run with the same dataset, the split will yield the same training and testing sets. This step is crucial in machine learning as it allows for a robust evaluation of the model's performance by testing it on unseen data.

Step 21 – Create a linear regression model

```
[151]: # Create a linear regression model
model = LinearRegression()
```

In this line of code, a linear regression model is instantiated using the `LinearRegression` class from the Scikit-learn library. This model will be used to analyze the relationship between the independent variable (x) and the dependent variable (y) from the dataset.

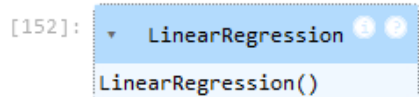
By creating an instance of the `LinearRegression` class, you can proceed to fit the model to the training data using the `fit()` method. Once fitted, the model can make predictions and evaluate

performance using metrics like mean squared error or (R^2) score. Linear regression is a foundational algorithm in machine learning, making it a great choice for understanding basic relationships in data.

Step 22 – Train the model

In this line of code, the linear regression model created in the previous step is trained using the training datasets `x_train` and `y_train`. The `fit()` method adjusts the model parameters to find the best-fitting line that minimizes the difference between the predicted values and the actual values of (y) .

```
[152]: # Train the model
       model.fit(x_train, y_train)
```

The image shows the Jupyter Notebook output for the training step. It displays the text 'LinearRegression' with a dropdown arrow on the left and two circular icons on the right. Below this, the text 'LinearRegression()' is visible, indicating the type of the object stored in the variable 'model'.

During this training process, the model learns the underlying relationship between the independent variable (x) and the dependent variable (y) . Once the model is trained, it can be used to make predictions on new data, allowing for analysis and insights into how changes in (x) affect (y) . This is a crucial step in building predictive models in machine learning.

Step 23 – Test the model

In this line of code, the trained linear regression model is used to make predictions on the testing dataset, `x_test`. The `predict()` method takes the features from the test set as input and outputs the predicted values of the dependent variable (y) , which are stored in the variable `y_pred`.

```
[153]: # Test the model
       y_pred = model.predict(x_test)
```

Testing the model is an essential step in the machine learning process, as it allows you to evaluate how well the model generalizes to unseen data. By comparing the predicted values (`y_pred`) with the actual values of (y) from the test set, you can assess the model's accuracy and effectiveness. This evaluation helps determine whether the model can make reliable predictions in real-world scenarios.

Step 24 – Print Results

```
[154]: # Print results
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R-squared:", r2_score(y_test, y_pred))

Mean Squared Error: 774.0998821054206
R-squared: 0.040060751916069504
```

This code evaluates the trained linear regression model's performance using two key metrics: Mean Squared Error (MSE) and R-squared (R^2). MSE calculates the average of the squared differences between the actual values and the predicted values, with lower values indicating better model accuracy. R^2 measures how much variance in the dependent variable is explained by the independent variable, with values closer to 1 suggesting a strong predictive relationship. Together, these metrics provide insights into the model's effectiveness in predicting outcomes.

Task 4 - Make Predictions

Step 25 – Value to predicate

This code snippet creates a DataFrame named `values\_to\_predict` containing three new values for the variable x: 120, 126, and 132. By organizing these values into a DataFrame, they can easily be used for making predictions with the previously trained linear regression model. This structure is essential for maintaining consistency in data formatting, allowing for seamless integration with the model's prediction methods.

```
[155]: # Values to predict
       values_to_predict = pd.DataFrame({'x': [120, 126, 132]})
```

Step 26 – Predictions

In this line of code, the trained linear regression model is used to make predictions based on the values provided in the `values\_to\_predict` DataFrame. The `model.predict()` function takes the new input data (in this case, the x-values of 120, 126, and 132) and computes the corresponding predicted y-values. These predictions are stored in the `predictions` variable, which can be utilized for further analysis or evaluation of the model's performance. This step is crucial in applying the model to new data to forecast outcomes based on the established relationship between x and y from the training dataset.

```
[156]: # Predictions
       predictions = model.predict(values_to_predict)
```

Step 27 – Print predication

This line of code prints the predicted results for the specified x-values of 120, 126, and 132. The `predictions` variable, which contains the model's output for these inputs, is displayed alongside a descriptive message. This allows you to easily see the forecasted y-values corresponding to the provided x-values, providing insight into the model's predictions based on the linear regression analysis conducted earlier. This output can be useful for interpreting how changes in x influence y according to the model.

```
[157]: print("Predicted Results for x=120, 126, 132:", predictions)
       Predicted Results for x=120, 126, 132: [51.76364566 51.90253396 52.04142225]
```

Step 28 – This code snippet visualizes the results of your linear regression model by creating a scatter plot and a regression line.

```
[180]: # Plotting predictions
plt.figure(figsize=(10, 6))

plt.scatter(df1['x'], df1['y'], color='blue', label='train data')
plt.scatter(values_to_predict['x'], predictions, color='red', label='Predictions')
plt.plot(df1['x'], model.predict(df1[['x']]), color='green', label='Regression Line')

plt.title('Linear Regression Model Predictions')

plt.xlabel('X-axis')
plt.ylabel('Y-axis')

plt.legend()

plt.show()
```

Figure Size - The plot's size is set to 10 inches wide and 6 inches tall for better visibility.

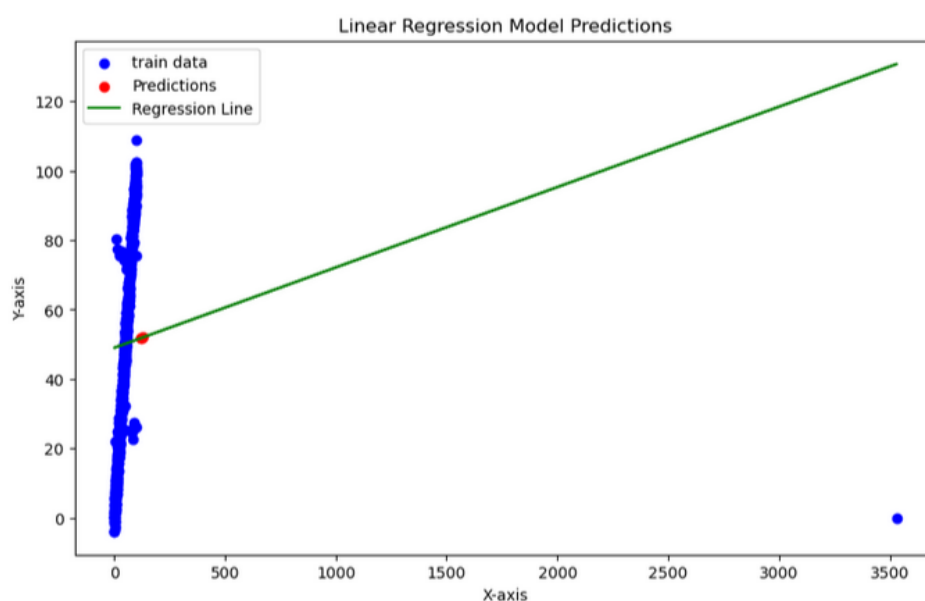
Data Points - The blue dots represent the original training data (`df1['x']` vs. `df1['y']`), while the red dots indicate the predictions made by the model for the specified x-values (120, 126, and 132).

Regression Line - The green line displays the fitted regression line for the training data, indicating the overall trend captured by the model.

Labels and Legend - The plot includes a title, axis labels for clarity, and a legend to distinguish between the training data, predictions, and the regression line.

Displaying the Plot - Finally, `plt.show()` renders the plot for visualization.

This graphical representation helps to assess how well the model predicts the y-values based on the provided x-values and illustrates the relationship between the variables.



Full Code

```
: # Import numpy and pandas Library
import numpy as np
import pandas as pd
```

```
: #Load the train dataset
df1=pd.read_csv("train.csv")
print(df1)
```

	x	y
0	NaN	NaN
1	NaN	NaN
2	24.0	21.549452
3	50.0	47.464463
4	15.0	17.218656
..	...	...
699	82.0	88.603770
700	66.0	63.648685
701	97.0	94.975266
702	100.0	93.456700
703	100.0	93.456700

```
#Load the test dataset
df2=pd.read_csv("test.csv")
print(df2)
```

	x	y
0	77	79.775152
1	21	23.177279
2	22	25.609262
3	20	17.857388
4	36	41.849864
..	..	...
295	71	68.545888
296	46	47.334876
297	55	54.090637
298	62	63.297171
299	47	52.459467

[300 rows x 2 columns]

```
#Check train.csv Missing Values
df1.isnull().sum()
```

```
x    2
```

```
: df1.loc[0,'x'] = 25 # Replace 'x' with the actual column name for x
df1.loc[0,'y'] = 23.456785 # Replace 'y' with the actual column name for y
df1.loc[1,'x'] = 56 # Assuming this is the second missing value for x
df1.loc[1,'y'] = 45.345665 # Assuming this is the second missing value for y

# Alternatively, you can use this if you know the exact index positions of missing values
# df1.loc[[index_1, index_2], ['x', 'y']] = [[24, 23.456785], [56, 45.345665]]

# Save the updated DataFrame (optional)
df1.to_csv('train_filled.csv', index=False)
```

```
# Display the updated DataFrame  
print(df1)
```

```
      x      y  
0  25.0 23.456785  
1  56.0 45.345665  
2  24.0 21.549452  
3  50.0 47.464463  
4  15.0 17.218656  
..   ...   ...  
699  82.0 88.603770  
700  66.0 63.648685  
701  97.0 94.975266  
702 100.0 93.456700  
703 100.0 93.456700
```

```
[704 rows x 2 columns]
```

```
# Drop train.csv Missing Value  
df3=df1.dropna()
```

```
df3.isnull().sum()
```

```
x      0  
y      0  
dtype: int64
```

```
#Check test.csv Missing Values  
df2.isnull().sum()
```

```
dtype: int64
```

```
#Check for Duplicate values  
df1.duplicated().sum()
```

```
1
```

```
# Drop Duplicate values  
df4=df1.drop_duplicates()
```

```
df4.duplicated().sum()
```

```
0
```

```
#Check for Duplicate values  
df2.duplicated().sum()
```

```
0
```

```
# Import Matplotlib
```

```
#train x  
x_train=df1["x"].values  
  
print(x_train)
```

```
[2.50000000e+01 5.60000000e+01 2.40000000e+01 5.00000000e+01
1.50000000e+01 3.80000000e+01 8.70000000e+01 3.60000000e+01
1.20000000e+01 8.10000000e+01 8.10000000e+01 5.00000000e+00
1.60000000e+01 1.60000000e+01 2.40000000e+01 3.90000000e+01
5.40000000e+01 6.00000000e+01 2.60000000e+01 7.30000000e+01
2.90000000e+01 3.10000000e+01 6.80000000e+01 8.70000000e+01
5.80000000e+01 5.40000000e+01 8.40000000e+01 5.80000000e+01
4.90000000e+01 2.00000000e+01 9.00000000e+01 4.80000000e+01
4.00000000e+00 4.00000000e+00 4.20000000e+01 0.00000000e+00
6.00000000e+01 9.30000000e+01 3.90000000e+01 7.00000000e+00
2.10000000e+01 6.80000000e+01 8.40000000e+01 0.00000000e+00
5.80000000e+01 1.90000000e+01 3.60000000e+01 1.90000000e+01
5.90000000e+01 5.10000000e+01 1.90000000e+01 3.30000000e+01
8.50000000e+01 4.40000000e+01 5.00000000e+00 5.90000000e+01
1.40000000e+01 9.00000000e+00 9.00000000e+00 6.90000000e+01
1.00000000e+01 1.70000000e+01 5.80000000e+01 7.40000000e+01
2.10000000e+01 5.10000000e+01 1.90000000e+01 5.00000000e+01]
```

```
#train y
y_train=df1["y"].values

print(y_train)
```

```
57.95782956 52.24760027 24.58286902 12.12573805 100.0158026
0. 74.04682658 1.61194747 70.36836307 52.26831735
83.1286166 43.64765048 49.44785426 72.6356699 52.78130641
57.11195136 79.1050629 101.6228548 53.5825402 68.92139297
46.9666961 51.02642868 85.52073551 99.51685756 94.63911256
46.78357742 21.21321959 58.37266004 87.22059677 102.4967859
43.88314335 53.06655757 26.33464785 98.52008934 9.40049758
52.94026699 53.83020877 45.94511142 65.0132736 86.5069584
75.63280796 36.78035027 100.5328916 29.04466136 51.70352433
9.19995472 71.70015848 49.82634062 37.49971096 53.65084683
33.92561965 49.92639685 8.14815426 49.72359037 16.16712757
75.30033002 9.57736857 48.38088357 72.95331671 92.59573853
88.85523586 99.00361771 45.09439571 46.94362684 48.33449605
94.92329574 47.78165248 81.28960746 37.83155021 39.69185252
76.92664854 88.02990531 56.99178872 72.58929383 44.98103442
11.99017641 1.91951333 1.62882607 66.27746655 57.53887255
```

```
# test x
x_test=df2["x"].values

print(x_test)
```

```
[ 77  21  22  20  36  15  62  95  20  5  4  19  96  62  36  15  65  14
 87  69  89  51  89  27  97  58  79  21  93  27  99  31  33  80  28  47
 53  69  28  33  91  71  50  76  4  37  70  68  40  35  94  88  52  31
 59  0  39  64  69  57  13  72  76  61  82  18  41  50  55  13  46  13
 79  53  15  28  81  69  52  84  68  27  56  48  40  39  82  100  59  43
 67  38  63  91  60  14  21  87  73  32  2  82  19  74  42  12  1  90
 89  0  41  16  94  97  66  24  17  90  13  0  64  96  98  12  41  47
 78  20  89  29  64  75  12  25  28  30  65  59  64  53  71  97  73  9
 12  63  99  60  35  2  60  32  94  84  63  22  81  93  33  7  42  46
 54  16  49  43  95  66  21  35  80  37  54  56  1  32  58  32  46  72
 17  97  93  91  37  4  54  51  27  46  92  73  77  91  61  99  4  72
 19  57  78  26  74  90  66  13  40  77  67  75  23  45  59  44  23  55
 55  95  12  4  7  100  48  42  96  39  100  87  14  14  37  5  88  91
 65  74  56  16  5  28  92  46  54  39  44  31  68  86  90  38  21  95
 56  60  65  78  89  6  67  36  16  100  45  73  57  20  76  34  55  72
 55  8  56  72  58  6  96  23  58  23  19  25  64  21  59  19  16  42
 43  61  92  11  41  1  8  71  46  55  62  47]
```

```

: # test y
y_test=df2["y"].values

print(y_test)

```

```

[ 79.77515201 23.17727887 25.60926156 17.85738813 41.84986439
  9.80523488 58.87465933 97.61793701 18.39512747  8.74674765
  2.81141583 17.09537241 95.14907176 61.38800663 40.24701716
 14.82248589 66.95806869 16.63507984 90.65513736 77.22982636
 92.11906278 46.91387709 89.82634442 21.71380347 97.41206981
 57.01631363 78.31056542 19.1315097  93.03483388 26.59112396
 97.55155344 31.43524822 35.12724777 78.61042432 33.07112825
 51.69967172 53.62235225 69.46306072 27.42497237 36.34644189
 95.06140858 68.16724757 50.96155532 78.04237454  5.60766487
 36.11334779 67.2352155  65.01324035 38.14753871 34.31141446
 95.28503937 87.84749912 54.08170635 31.93063515 59.61247085
 -1.04011421 47.49374765 62.60089773 70.9146434  56.14834113
 14.05572877 68.11367147 75.59701346 59.225745  85.45504157
 17.76197116 38.68888682 50.96343637 51.83503872 17.0761107
 46.56141773 10.34754461 77.91032969 50.17008622 13.25690647
 31.32274932 73.9308764  74.45114379 52.01932286 83.68820499
 70.3698748  23.44479161 49.83051801 49.88226593 41.04525583
 33.37834391 81.29750133 105.5918375  56.82457013 48.67252645
 67.02150613 38.43076389 58.61466887 89.12377509 60.9105427
 13.83959878 16.89085185 84.06676818 70.34969772 33.38474138
 -1.63296825 88.54475895 17.44047622 75.69298554 41.97607107
 12.59244741  0.27530726 98.13258005 87.45721555 -2.34473854

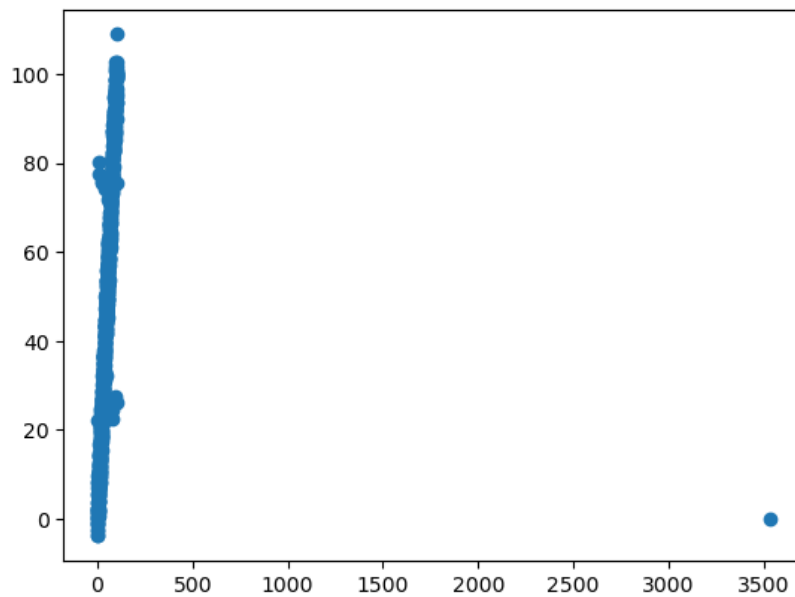
```

```

# Scatter Plot
plt.scatter(x_train, y_train)

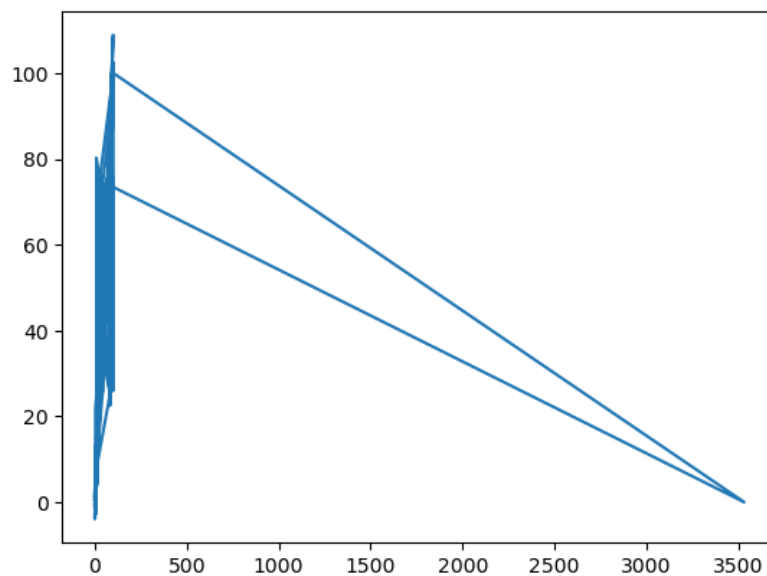
plt.show()

```



```
# Line Plot
plt.plot(x_train, y_train)

plt.show()
```



```
: from sklearn.model_selection import train_test_split
```

```
: from sklearn.linear_model import LinearRegression
```

```
: from sklearn.metrics import mean_squared_error, r2_score
```

```
: # Prepare the data
```

```
x = df1[['x']]
```

```
y = df1['y']
```

```
: # Split the data into training and testing sets
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
: # Create a linear regression model
```

```
model = LinearRegression()
```

```
: # Train the model
```

```
model.fit(x_train, y_train)
```

```
: ▼ LinearRegression ⓘ ⓘ
```

```
LinearRegression()
```

```
: # Print results
```

```
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

```
print("R-squared:", r2_score(y_test, y_pred))
```

```
Mean Squared Error: 774.0998821054206
```

```
R-squared: 0.040060751916069504
```

```
: # Values to predict
```

```
values_to_predict = pd.DataFrame({'x': [120, 126, 132]})
```

```
# Predictions
predictions = model.predict(values_to_predict)
```

```
print("Predicted Results for x=120, 126, 132:", predictions)
```

```
Predicted Results for x=120, 126, 132: [51.76364566 51.90253396 52.04142225]
```

```
# Plotting predictions
plt.figure(figsize=(10, 6))

plt.scatter(df1['x'], df1['y'], color='blue', label='train data')
plt.scatter(values_to_predict['x'], predictions, color='red', label='Predictions')
plt.plot(df1['x'], model.predict(df1[['x']]), color='green', label='Regression Line')

plt.title('Linear Regression Model Predictions')

plt.xlabel('X-axis')
plt.ylabel('Y-axis')

plt.legend()

plt.show()
```

```
plt.show()
```

